

Software psychology: poznawcze uwarunkowania interakcji człowieka z oprogramowaniem

dr inż. Mateusz Pomianek

Abstrakt

Software psychology jest historycznym określeniem interdyscyplinarnego obszaru badań nad zachowaniem człowieka podczas tworzenia, uczenia się i użytkowania systemów komputerowych. W ujęciu zaproponowanym przez Shneidermana (1980) centralnym przedmiotem badań jest funkcjonowanie człowieka podczas korzystania z systemów komputerowych i informacyjnych, przy czym zasadniczą rolę odgrywają mechanizmy poznawcze i percepcyjne, a nie wyłącznie fizyczna ergonomia stanowiska pracy. Współcześnie zakres dawnej *software psychology* został w znacznej mierze włączony do interakcji człowiek-komputer (*human-computer interaction* [HCI]), ergonomii poznawczej, inżynierii poznawczej, psychologii programowania oraz badań nad doświadczeniem użytkownika (*user experience* [UX]). W niniejszym opracowaniu przedstawiono historyczne źródła tej dziedziny, jej relacje z obszarami pokrewnymi, podstawowe mechanizmy poznawcze związane z użytkowaniem oprogramowania, modele teoretyczne, metody badań empirycznych oraz ich zastosowania w projektowaniu współczesnych systemów interaktywnych, w tym systemów wykorzystujących sztuczną inteligencję. Szczególną uwagę poświęcono uwadze, pamięci roboczej, obciążeniu poznawczemu, modelom mentalnym, podejmowaniu decyzji, powstawaniu błędów oraz zjawiskom związanym z automatyzacją i kalibracją zaufania.

Słowa kluczowe: software psychology, HCI, ergonomia poznawcza, modele mentalne, obciążenie poznawcze, interakcja człowiek-komputer, sztuczna inteligencja

Wprowadzenie

Rozwój informatyki przez wiele lat koncentrował się przede wszystkim na właściwościach technicznych systemów: wydajności obliczeniowej, organizacji pamięci, algorytmach oraz poprawności programów. Wraz z upowszechnieniem komputerów coraz wyraźniej ujawniała się jednak potrzeba analizowania oprogramowania jako środowiska ludzkiej aktywności.

Weinberg (1971) należał do pierwszych badaczy, którzy systematycznie argumentowali, że programowanie należy analizować nie tylko jako problem techniczny, lecz także jako formę ludzkiego zachowania. W książce *The Psychology of Computer Programming* rozpatrywał m.in. różnice indywidualne między programistami, organizację zespołów, motywację, uczenie się oraz wpływ języków programowania na sposób rozwiązywania problemów.

Bardziej ogólną koncepcję przedstawił Shneiderman (1980) w pracy *Software Psychology: Human Factors in Computer and Information Systems*. *Software psychology* została przez niego potraktowana jako obszar wykorzystujący psychologię eksperymentalną, psychologię poznawczą, psycholingwistykę oraz badania nad czynnikami ludzkimi do zrozumienia funkcjonowania użytkowników systemów komputerowych.

Termin *software psychology* ma obecnie przede wszystkim znaczenie historyczne. Nie powstała jedna wyraźnie wyodrębniona dyscyplina funkcjonująca współcześnie pod tą nazwą. Jej problematyka została włączona przede wszystkim do HCI, ergonomii poznawczej, psychologii programowania, *cognitive engineering* i UX.

Software psychology, a dziedziny pokrewne

Interakcja człowiek-komputer

Najbliższym współczesnym odpowiednikiem *software psychology* jest HCI. Interakcja człowiek-komputer obejmuje projektowanie, implementację i empiryczną ocenę systemów interaktywnych oraz badanie zachowania ludzi podczas ich użytkowania.

Zakres HCI jest jednak szerszy. Obejmuje nie tylko poznanie, ale również czynniki społeczne, organizacyjne, emocjonalne, kulturowe oraz fizyczne. *Software psychology* w historycznym ujęciu Shneidermana (1980) kładła natomiast szczególny nacisk na poznawcze uwarunkowania pracy z oprogramowaniem.

Ergonomia poznawcza

Ergonomia poznawcza bada dopasowanie systemu do właściwości procesów takich jak percepcja, pamięć, uwaga, podejmowanie decyzji i rozwiązywanie problemów. Pod tym względem jej zakres w dużej mierze odpowiada głównym postulatom *software psychology*.

W przeciwieństwie do ergonomii fizycznej problemem nie jest przede wszystkim wysokość biurka czy biomechanika ruchu, lecz sposób organizacji informacji, przewidywalność interfejsu, liczba elementów wymagających zapamiętania oraz zgodność systemu z modelem mentalnym użytkownika.

Inżynieria poznawcza

Inżynieria poznawcza (*cognitive engineering*) analizuje relację pomiędzy strukturą systemu technicznego, a mechanizmami poznawczymi operatora. Norman (1986) wskazywał, że projektowanie powinno minimalizować rozbieżność pomiędzy intencją człowieka a operacjami wymaganymi przez system.

Z tej perspektywy interfejs jest nie tyle powierzchnią graficzną, ile narzędziem pośredniczącym między reprezentacją problemu w umyśle użytkownika a reprezentacją wykorzystywaną przez system.

Psychologia programowania

Psychologia programowania koncentruje się na poznawczych i społecznych procesach związanych z tworzeniem programów komputerowych. Obejmuje m.in. rozumienie kodu, debugowanie, projektowanie algorytmów, reprezentację problemów oraz wpływ języków programowania na rozumowanie.

Stanowi więc bardziej wyspecjalizowaną część problematyki rozwijanej pierwotnie przez Weinberga (1971) i Shneidermana (1980).

User experience

UX koncentruje się na całościowym doświadczeniu korzystania z systemu, obejmującym użyteczność, emocje, oczekiwania, estetykę i subiektywną wartość produktu. Nie jest to zatem synonim *software psychology*. Psychologia poznawcza dostarcza natomiast podstaw teoretycznych wielu metod wykorzystywanych w badaniach UX.

Mechanizmy poznawcze w interakcji z oprogramowaniem

Uwaga

Uwaga jest zasobem ograniczonym. System informatyczny konkuruje o zasoby uwagowe użytkownika z wykonywanym zadaniem, otoczeniem oraz innymi systemami.

Interfejs może wspierać selekcję informacji poprzez hierarchię wizualną, grupowanie, odpowiednie wykorzystanie przestrzeni i sygnałów wyróżniających. Może również ją zakłócać poprzez nadmiar powiadomień, animacji, komunikatów i konkurujących elementów.

Problemy te są szczególnie istotne w systemach wielozadaniowych, środowiskach kontroli procesów oraz systemach bezpieczeństwa krytycznego.

Pamięć robocza

Pamięć robocza umożliwia czasowe przechowywanie i przetwarzanie informacji, jednak jej pojemność jest ograniczona (Baddeley, 2000). Projektowanie interfejsu wymagającego zapamiętywania wielu niezależnych informacji zwiększa prawdopodobieństwo błędów.

Z tego powodu projekt powinien preferować rozpoznawanie zamiast przypominania. Informacje potrzebne do podjęcia decyzji powinny pozostawać dostępne percepcyjnie, jeżeli nie istnieje uzasadniona potrzeba ich zapamiętywania.

Pamięć długotrwała i uczenie się

Doświadczeni użytkownicy nie wykonują zadań w taki sam sposób jak osoby początkujące. Wraz z uczeniem się powstają schematy poznawcze umożliwiające grupowanie informacji i automatyzację części operacji.

Konsekwencją projektową jest potrzeba zapewnienia zarówno łatwej ścieżki dla początkujących, jak i wydajnych mechanizmów dla ekspertów, np. skrótów klawiaturowych, makr lub operacji wsadowych.

Obciążenie poznawcze

Teoria obciążenia poznawczego opisuje ograniczenia zasobów przetwarzania dostępnych podczas wykonywania zadania (Sweller, 1988). Oprogramowanie może generować obciążenie wynikające zarówno z samego problemu, jak i ze sposobu reprezentowania informacji.

Projektant powinien zatem ograniczać działania, które nie służą realizacji celu użytkownika. Szczególnie problematyczne są zbędne kroki, niejasne etykiety, rozproszone informacje oraz konieczność ciągłego przełączania kontekstu.

Minimalizm interfejsu nie powinien jednak oznaczać automatycznego usuwania informacji. Zbyt silna redukcja widocznych danych może zwiększyć obciążenie pamięci i utrudnić tworzenie poprawnego modelu mentalnego.

Modele mentalne

Użytkownik tworzy wewnętrzną reprezentację działania systemu. Model taki nie musi być technicznie poprawny. Wystarczy, aby umożliwiał przewidywanie konsekwencji działań.

Problemy pojawiają się, gdy model użytkownika znacznie różni się od rzeczywistego modelu systemu. Interfejs powinien więc przekazywać informacje pozwalające odpowiedzieć co najmniej na trzy pytania:

1. Jaki jest aktualny stan systemu?
2. Co stanie się po wykonaniu określonego działania?
3. Dlaczego system zachował się w określony sposób?

Dobra informacja zwrotna nie pełni zatem wyłącznie funkcji informacyjnej. Jest mechanizmem budowania modelu mentalnego.

Percepcja i reprezentacja informacji

Percepcja wizualna umożliwia bardzo szybkie wykrywanie części właściwości bodźców. Odpowiednio zaprojektowana reprezentacja może więc przenosić część obciążenia z wymagającego świadomego rozumowania na procesy percepcyjne.

Przykładem jest wizualizacja wartości poprzez położenie na wspólnej osi zamiast wymagania od użytkownika porównywania liczb w tabeli.

Nie oznacza to jednak, że graficzna forma reprezentacji jest zawsze lepsza. Dobór sposobu prezentacji powinien zależeć od zadania.

Podejmowanie decyzji

Systemy komputerowe często nie służą bezpośrednio wykonywaniu operacji, lecz wspomaganiu decyzji. Ich jakość powinna być więc oceniana również pod kątem wpływu na proces decyzyjny.

Istotnymi problemami są m.in. nadmiar informacji, niepewność, sposób przedstawiania ryzyka oraz heurystyki i błędy poznawcze.

Projekt może zmieniać decyzje użytkownika nawet wtedy, gdy formalnie przedstawia te same dane. Sposób prezentacji informacji staje się więc częścią architektury decyzyjnej.

Błędy użytkownika

Reason (1990) rozróżniał m.in. błędy wynikające z nieprawidłowego zamiaru oraz błędy podczas realizacji poprawnej intencji.

Z punktu widzenia projektowania ważne jest odejście od interpretowania pomyłek wyłącznie jako „błędu użytkownika”. Powtarzalny błąd często wskazuje na systematyczną niezgodność pomiędzy sposobem funkcjonowania systemu a sposobem rozumowania człowieka.

System powinien więc:

- zapobiegać przewidywalnym błędom;
- umożliwiać bezpieczne cofnięcie operacji;
- wyraźnie komunikować konsekwencje;
- wykrywać stany nieprawidłowe;
- wspomagać odzyskiwanie po błędzie.

Teoretyczne modele interakcji

Model Human Processor

Card, Moran i Newell (1983) przedstawili człowieka jako system przetwarzania informacji złożony z podsystemów percepcyjnych, poznawczych i motorycznych.

Model umożliwiał ilościowe przewidywanie czasu wykonania prostych zadań interakcyjnych i stworzył podstawy m.in. rodziny metod GOMS.

GOMS

GOMS opisuje zadanie za pomocą:

- *Goals* — celów;
- *Operators* — podstawowych operacji;
- *Methods* — metod realizacji celu;
- *Selection rules* — reguł wyboru pomiędzy metodami.

Modele tego typu pozwalają przewidywać zachowanie doświadczonych użytkowników wykonujących dobrze wyuczone, rutynowe zadania.

Ich ograniczeniem jest słaba reprezentacja eksploracji, uczenia się, emocji, niepewności i błędów.

Model siedmiu etapów działania

Norman (1986) opisał interakcję jako cykl obejmujący sformułowanie celu, intencji, planu działania, wykonanie operacji, percepcję stanu systemu, interpretację i ocenę wyniku.

Model ten prowadzi do dwóch kluczowych pojęć: *gulf of execution* oraz *gulf of evaluation*. Dobry interfejs powinien minimalizować zarówno trudność przełożenia zamiaru na działanie, jak i trudność zrozumienia reakcji systemu.

Metody badań software psychology

Badanie poznawczych aspektów interakcji wymaga triangulacji metod.

Eksperymenty kontrolowane

Eksperymenty pozwalają porównywać warianty interfejsów przy kontrolowanych warunkach.

Typowymi zmiennymi zależnymi są:

- czas wykonania zadania;
- liczba błędów;
- odsetek poprawnie zakończonych zadań;
- liczba kroków;
- liczba korekt;

- czas uczenia się.

Eye tracking

Śledzenie wzroku pozwala badać rozmieszczenie uwagi wizualnej. Analizowane mogą być m.in. liczba i czas fiksacji, sakady, czas do pierwszej fiksacji oraz przejścia pomiędzy obszarami zainteresowania.

Eye tracking nie umożliwia jednak bezpośredniego wnioskowania o procesach poznawczych. Fakt patrzenia na element nie dowodzi jego zrozumienia.

Metody psychofizjologiczne

Badania HCI coraz częściej wykorzystują EEG, EDA, ECG, PPG, pupillometrię i inne sygnały fizjologiczne do pośredniego szacowania obciążenia, pobudzenia i uwagi.

Ich interpretacja wymaga jednak ostrożności, ponieważ podobna reakcja fizjologiczna może mieć wiele przyczyn.

Protokoły think-aloud

Uczestnik werbalizuje myśli podczas wykonywania zadania. Metoda pozwala identyfikować strategie, nieporozumienia oraz elementy systemu powodujące trudności.

Werbalizacja może jednak zmieniać przebieg wykonywania zadania i nie zapewnia pełnego dostępu do automatycznych procesów poznawczych.

Kwestionariusze

Do często wykorzystywanych miar należą oceny użyteczności, obciążenia, zaufania i doświadczenia użytkownika. Dane samoopisowe powinny być jednak traktowane jako uzupełnienie, a nie zamiennik pomiarów behawioralnych.

W szczególności subiektywna łatwość systemu nie musi oznaczać większej skuteczności.

Automatyzacja i systemy sztucznej inteligencji

Rozwój systemów wykorzystujących AI ponownie zwiększył znaczenie psychologicznego podejścia do interakcji.

Zaufanie do automatyzacji

Zaufanie użytkownika powinno odpowiadać rzeczywistym możliwościom systemu. Zarówno niedostateczne, jak i nadmierne zaufanie może pogarszać jakość decyzji (Lee & See, 2004; Parasuraman & Riley, 1997).

Celem projektowania nie jest więc maksymalizacja zaufania, lecz jego kalibracja.

Automation bias

Użytkownik może bezkrytycznie zaakceptować rekomendację systemu tylko dlatego, że pochodzi ona z systemu automatycznego. Problem staje się szczególnie istotny w diagnostyce, medycynie, transporcie i systemach decyzyjnych.

Explainable AI

Wyjaśnienie działania modelu AI powinno odpowiadać potrzebom poznawczym użytkownika. Większa ilość informacji nie musi prowadzić do lepszego rozumienia.

Wyjaśnienia mogą zwiększać zaufanie również wtedy, gdy rekomendacja systemu jest błędna. Z tego względu skuteczność *explainable AI* powinna być oceniana poprzez rzeczywistą zdolność użytkownika do wykrywania błędów, przewidywania zachowania systemu i podejmowania lepszych decyzji, a nie wyłącznie poprzez deklarowane poczucie zrozumienia.

Generatywna sztuczna inteligencja

Systemy generatywne tworzą szczególny problem poznawczy, ponieważ ich odpowiedzi często mają wysoką płynność językową niezależnie od poprawności treści.

Może to prowadzić do błędnej heurystyki:

płynność językowa → kompetencja → prawdziwość.

Z perspektywy *software psychology* interfejs systemu generatywnego powinien zatem wspierać monitorowanie epistemiczne użytkownika poprzez sygnalizowanie niepewności, udostępnianie źródeł, umożliwianie weryfikacji oraz wyraźne odróżnianie danych od wniosków modelu.

Znaczenie software psychology dla współczesnego projektowania

Najważniejszym założeniem psychologicznego podejścia do oprogramowania jest traktowanie interfejsu jako części systemu poznawczego człowieka.

Dobrze zaprojektowane oprogramowanie nie tylko umożliwia wykonanie określonego zadania. Powinno również:

- redukować zbędne obciążenie pamięci roboczej;
- kierować uwagę na informacje istotne;
- wspierać prawidłowy model mentalny;
- umożliwiać przewidywanie skutków działania;
- dostarczać jednoznacznej informacji zwrotnej;
- ograniczać skutki przewidywalnych błędów;
- wspierać uczenie się;
- umożliwiać stopniowe przechodzenie od działania kontrolowanego do eksperckiego;
- wspierać metapoznanie i ocenę niepewności.

W takim ujęciu jakość oprogramowania nie jest wyłącznie właściwością kodu. Jest właściwością relacji pomiędzy systemem, zadaniem, użytkownikiem i kontekstem.

Ograniczenia i problemy metodologiczne

Jednym z podstawowych problemów badań poznawczych nad oprogramowaniem jest ograniczona generalizacja wyników eksperymentów laboratoryjnych. Zadania trwające kilka minut nie zawsze reprezentują wielogodziną lub wieloletnią pracę z rzeczywistym systemem.

Drugim problemem jest ogromne zróżnicowanie użytkowników. Wiedza domenowa, wiek, doświadczenie, zdolności poznawcze, język oraz wcześniejsze modele mentalne mogą silnie moderować efekt określonego projektu.

Trzecim problemem jest redukcjonizm. Próba sprowadzenia interakcji do czasu reakcji, liczby kliknięć lub pojemności pamięci roboczej może pomijać kontekst społeczny, emocjonalny i organizacyjny.

Dlatego współczesne HCI coraz częściej łączy eksperymenty ilościowe, badania jakościowe, analizę danych telemetrycznych oraz badania prowadzone w naturalnym środowisku użytkowników.

Kierunki dalszych badań

Szczególnie istotne pozostają następujące problemy:

1. poznawcze konsekwencje współpracy człowieka z generatywną AI;
2. kalibracja zaufania do systemów probabilistycznych;
3. wpływ AI na metapoznanie i poczucie własnej wiedzy;
4. *cognitive offloading*, czyli przenoszenie części procesów poznawczych do systemów cyfrowych;
5. zmiany kompetencji wynikające z długotrwałej automatyzacji;
6. projektowanie systemów adaptujących się do stanu poznawczego użytkownika;
7. uwzględnianie neuroróżnorodności i różnic indywidualnych;
8. projektowanie wyjaśnień dostosowanych do celu, wiedzy i kompetencji odbiorcy;
9. pomiar długoterminowych konsekwencji interakcji człowiek-AI.

Szczególnie interesująca jest zmiana relacji między człowiekiem, a systemem. W klasycznych interfejsach użytkownik wydawał polecenia deterministycznemu narzędziu. W systemach generatywnych coraz częściej prowadzi dialog z probabilistycznym agentem, którego zachowania nie można całkowicie przewidzieć.

Wymaga to rozszerzenia tradycyjnych modeli HCI o procesy takie jak ocena wiarygodności, monitorowanie epistemiczne, negocjowanie intencji oraz dynamiczna kalibracja zaufania.

Wnioski

Software psychology należy traktować przede wszystkim jako ważny historycznie etap rozwoju badań nad interakcją człowieka z komputerem, a nie jako dominującą współcześnie nazwę samodzielnej dyscypliny. Jej podstawowe założenia pozostają jednak aktualne.

Najważniejszym z nich jest przekonanie, że oprogramowanie należy projektować z uwzględnieniem ograniczeń i właściwości ludzkiego systemu poznawczego.

Współczesna kontynuacja tej perspektywy znajduje się przede wszystkim w HCI, ergonomii poznawczej, inżynierii poznawczej, psychologii programowania i badaniach UX. Rozwój systemów wykorzystujących sztuczną inteligencję ponownie zwiększa znaczenie tych zagadnień. Kluczowym problemem nie jest już wyłącznie to, czy użytkownik potrafi obsłużyć system, lecz czy potrafi prawidłowo zrozumieć jego działanie, ograniczenia, stopień niepewności i wiarygodność generowanych wyników.

W tym sensie podstawowe pytanie *software psychology* pozostaje niezmienione: **w jaki sposób właściwości systemu informatycznego oddziałują z właściwościami ludzkiego systemu poznawczego i jak projektować tę interakcję, aby zwiększać skuteczność, bezpieczeństwo oraz jakość rozumowania człowieka?**

Bibliografia

Amershi, S., Weld, D., Vorvoreanu, M., Fourney, A., Nushi, B., Collisson, P., Suh, J., Iqbal, S., Bennett, P. N., Inkpen, K., Teevan, J., Kikin-Gil, R., & Horvitz, E. (2019). Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (pp. 1-13). Association for Computing Machinery. <https://doi.org/10.1145/3290605.3300233>

Baddeley, A. (2000). The episodic buffer: A new component of working memory? *Trends in Cognitive Sciences*, 4(11), 417-423. [https://doi.org/10.1016/S1364-6613\(00\)01538-2](https://doi.org/10.1016/S1364-6613(00)01538-2)

Card, S. K., Moran, T. P., & Newell, A. (1983). *The psychology of human-computer interaction*. Lawrence Erlbaum Associates.

Endsley, M. R. (1995). Toward a theory of situation awareness in dynamic systems. *Human Factors*, 37(1), 32-64. <https://doi.org/10.1518/001872095779049543>

Hoff, K. A., & Bashir, M. (2015). Trust in automation: Integrating empirical evidence on factors that influence trust. *Human Factors*, 57(3), 407-434. <https://doi.org/10.1177/0018720814547570>

Lee, J. D., & See, K. A. (2004). Trust in automation: Designing for appropriate reliance. *Human Factors*, 46(1), 50-80. https://doi.org/10.1518/hfes.46.1.50_30392

Norman, D. A. (1986). Cognitive engineering. In D. A. Norman & S. W. Draper (Eds.), *User centered system design: New perspectives on human-computer interaction* (pp. 31-61). Lawrence Erlbaum Associates.

Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2), 230-253. <https://doi.org/10.1518/001872097778543886>

Reason, J. (1990). *Human error*. Cambridge University Press.

Shneiderman, B. (1980). *Software psychology: Human factors in computer and information systems*. Winthrop Publishers.

Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257-285. https://doi.org/10.1207/s15516709cog1202_4

Weinberg, G. M. (1971). *The psychology of computer programming*. Van Nostrand Reinhold.